



Code review — variante bac

Runde de review pe proiectul tău, cea mai nouă sus.

Un singur fișier, runda cea mai nouă sus. Rundele vechi rămân dedesubt.

Runda 1 — 2026-09-15 — sha `46a53db`

Commit `46a53db` `tema` . Fișiere noi: `varianta3.h` , `varianta5.h` , `varianta24.h` , `varianta39.h` .
Restul (din `4281854` `1st push`) a fost verificat integral, pentru că e prima rundă pe repo-ul ăsta.

Codul NU a fost modificat — totul e doar aici.

Verificat prin **compilare și rulare**, nu prin citire: `g++ -std=c++17 -Wall -Wextra` , fiecare header compilat separat, apoi rulat cu exemplele din enunțuri.

Verdictul pe scurt.

Poarta de compilare: trecută. Toate cele 14 headere compilează. Asta e o veste bună și e prima dată când o scriu la tine fără „dar nu compilează”. Au ieșit însă 4 warning-uri, iar 3 dintre ele sunt bug-uri reale, nu zgomot.

Din cele 4 fișiere ale temei, unul singur produce răspunsul cerut.

fișier	ce face la rulare	verdict
<code>varianta39.h</code>	DA / NU corect pe ambele cazuri	✅ (o rezervă, M4)
<code>varianta3.h</code>	pierde cifra 9, nu afișează niciodată <code>0</code>	❌ B4, B5
<code>varianta5.h</code>	afișează mereu „nu exista numere prime”	❌ B6
<code>varianta24.h</code>	nu afișează nimic	❌ B1, B2, B3

Tiparul runde: **te oprești la un pas de final.** În `varianta5.h` ai scris `f`, ai scris sortarea, ai scris afișarea — și ai sărit peste singura linie care le leagă (`if (f(x) == x) ...`). În `varianta24.h` ai calculat `suma_ramasa` și `elemente_ramase` și n-ai scris `cout`. În `varianta3.h` ai calculat `imp` și `mij` și nu le-ai folosit. Nu e necunoaștere — e cod abandonat cu două linii înainte de linia de sosire. Compilatorul ți-a și spus: „*variable 'imp' set but not used*”.

Un lucru bun, real: algoritmi pe care i-ai dus până la capăt sunt corecți pe datele din enunț — `varianta1` (b), `7`, `12`, `17`, `20`. Verificate prin rulare, nu presupuse.

Ce am rulat, ca să vezi dovada

```
V3 : n=21523531 -> 53211235 (corect, din noroc – vezi B4)
V3 : n=12272351 -> 2112 (trebuia 0)
V3 : n=11009900 -> 1001 (trebuia 90011009 – cei doi de 9 au
                        dispărut complet)
V5 : 7 / 1125 2 314 101 37 225 12 -> "nu exista numere prime"
                        (trebuia "2 37 101")
V24: 5 / 1 9 4 8 5 -> (nimic) (trebuia 5.667)
V39: 5 / 4 3 6 1 8 -> DA      corect
V39: 5 / 8 3 6 1 4 -> NU      corect
V8 : (nimic)          (trebuia şirul alternat)
```

Critice

B1 — parametri de ieșire transmiși prin copie

varianta24.h:18

CUM E ACUM: `p(x, n, mini, maxi, sum)` se execută, calculează corect minimul, maximul și suma — și le pierde pe toate trei. În `solutie2b`, după apel, `mini`, `maxi` și `sum` au exact aceleași valori nedefinite ca înainte. Compilatorul te-a avertizat: „*variable 'mini' is uninitialized when used here*”.

DE CE: în C++, un parametru obișnuit e o **copie**. Funcția `p` primește propriile ei `mini`, `maxi`, `sum`, scrie în ele, iar la `}` copiile dispar. Variabilele din `solutie2b` n-au fost niciodată atinse.

Și atunci de ce ți se pare că tabloul `x` „se transmite înapoi”? Pentru că tablourile sunt excepția: `int x[]` ca parametru nu e un tablou, e un **pointer** la primul element al tabloului original. Când scrii `x[i] = ...` într-o funcție, scrii direct în memoria apelantului. De aici ți-a venit presupunerea că și restul parametrilor se comportă la fel — dar `int mini` e o copie curată.

Ca să întorci o valoare prin parametru, ceri compilatorului să nu mai copieze: `int &mini`. Ampersand-ul înseamnă „aceeași cutie de memorie, alt nume”.

```
// ACUM
void p(int x[], int n, int mini, int maxi, int sum) {

// CORECT
void p(int x[], int n, int &mini, int &maxi, int &sum) {
```

B2 — `for (int i = i; ...)` — bucla pornește de la un număr aleatoriu

varianta24.h:23

CUM E ACUM: warning la compilare: „*variable 'i' is uninitialized when used within its own initialization*”. Nu e eroare, deci programul se construiește și rulează.

DE CE: în `int i = i`, variabila `i` e considerată **deja declarată** în momentul în care se evaluează partea dreaptă. Deci nu copiezi `i` -ul de altundeva — îți citești propria variabilă înainte să fi pus ceva în ea. Valoarea e ce se întâmplă să fie în memoria aia: poate 0, poate 32764, poate negativ.

Consecința nu e „un rezultat greșit”, e imprevizibilă: dacă iese un număr mare, bucla nu rulează deloc (`i < n` fals din start) și minimul rămâne `x[0]`; dacă iese negativ, citești `x[-5]` — memorie care nu-ți aparține. Același program, două rulări, două rezultate.

```
// ACUM
for (int i = i; i < n; i++) {

// CORECT
for (int i = 1; i < n; i++) {
```

`1`, nu `0`: `x[0]` e deja pus în `mini` / `maxi` / `sum` pe liniile 19-21.

B3 — programul nu afișează nimic, iar media s-ar calcula în întregi

varianta24.h:57-58

CUM E ACUM: rulat cu `5` și `1 9 4 8 5`, programul citește, apelează `p`, calculează două variabile — și se termină. Ecran gol. Warning: „unused variable 'suma_ramasa'”.

DE CE: al doilea bug e ascuns în spatele primului și te-ar fi lovit imediat după ce puneai `cout`.

`suma_ramasa` și `elemente_ramase` sunt amândouă `int`. În C++, `/` între doi `int` e **împărțire întreagă**: rezultatul se trunchiază, nu se rotunjește. `17 / 3` dă `5`, nu `5.667` — zecimalele nu se pierd la afișare, nu se calculează niciodată.

Ca să obții zecimale, cel puțin un operand trebuie să fie real. Iar enunțul cere „cel mult 3 cifre după virgulă” — asta se cere explicit cu `fixed` + `setprecision(3)` din `<iomanip>`, altfel `cout` îți dă 6 cifre semnificative.

```
// ACUM
int suma_ramasa = sum - mini - maxi;
int elemente_ramase = n - 2;

// CORECT
double media = (double)(sum - mini - maxi) / (n - 2);
cout << fixed << setprecision(3) << media;
```

B4 — cifra 9 nu e numărată niciodată

`varianta3.h:48` (+ același bug la `varianta3.h:62`)

CUM E ACUM: `n = 11009900` ar trebui să dea `90011009` . Dă `1001` . Cei doi de 9 au dispărut din răspuns.

DE CE: `fr` are 10 căsuțe, indexate `0..9` . Bucla ta merge `i < 9` , deci se oprește la `i = 8` . `fr[9]` nu e scris niciodată.

Și aici e partea care contează: `fr[9]` nu e `0` . Un tablou local `int fr[10]` **nu se inițializează automat** — conține ce era în memoria aia de la codul rulat înainte. Bucla de afișare pornește tocmai de la `i = 9` și citește `fr[9] / 2` — gunoi. La rulările mele gunoiul a fost mic și cifra 9 pur și simplu a lipsit; la altă rulare ar putea tipări 9 de zeci de ori.

Exemplul din enunț (`21523531`) îți iese corect **doar pentru că nu conține cifra 9**. De asta datele de test trebuie alese ca să lovească bug-ul — exact lucrul pe care l-ai făcut bine la runda 8 pe `victor-vectori` .

```
// ACUM
for (int i = 0; i < 9; i++) {      // linia 48
    ...
for (int i = 0; i < 9; i++) {      // linia 62

// CORECT
int fr[10] = { 0 };              // linia 44, plasa de siguranță
for (int i = 0; i <= 9; i++) {
    ...
for (int i = 0; i <= 9; i++) {
```

B5 — palindromul: 0 nu se afișează niciodată, zeroul lipsește din prima jumătate, mijlocul lipsește

`varianta3.h:50-66`

CUM E ACUM: `n = 12272351` trebuia să dea `0` (nu se poate face palindrom). Dă `2112` . `n = 11009900` trebuia `90011009` ; dă `1001` .

DE CE: ai calculat corect ce-ți trebuia și n-ai folosit nimic. Compilatorul ți-a spus asta de două ori: „*imp' set but not used*”, „*mij' set but not used*”. Sunt trei lipsuri într-un singur algoritm:

1. **Testul de imposibilitate.** Un palindrom se citește la fel în ambele sensuri, deci fiecare cifră trebuie să apară de un număr **par** de ori — singura excepție e cifra din mijloc, care există doar la lungimi impare. `imp` numără exact cifrele cu frecvență impară. Regula: dacă lungimea e pară și `imp != 0`, sau dacă `imp > 1`, răspunsul e `0`. Tu nu te uiți deloc la el, așa că afișezi jumătatea unui palindrom care nu există.
2. **Zeroul din prima jumătate.** Prima buclă merge `i = 9; i > 0`, deci `0` nu intră în jumătatea descrescătoare. Pentru `11009900` răspunsul corect e `90011009` — zeroul e chiar acolo.
3. **Cifra din mijloc.** La lungime impară, cifra rămasă (`mij`) se scrie între cele două jumătăți. Ai reținut-o în `mij` și n-ai tipărit-o.

```
// ACUM
for (int i = 9; i > 0; i--) {           // sare peste 0
    for (int j = 1; j <= fr[i] / 2; j++) cout << i;
}
for (int i = 0; i < 9; i++) { ... }    // sare peste 9

// CORECT
if (imp > 1) { cout << 0; return; }
for (int i = 9; i >= 0; i--)
    for (int j = 1; j <= fr[i] / 2; j++) cout << i;
if (imp == 1) cout << mij;
for (int i = 0; i <= 9; i++)
    for (int j = 1; j <= fr[i] / 2; j++) cout << i;
```

B6 — punctul b) nu e scris: `f` nu e apelat niciodată

`varianta5.h:44-46`

CUM E ACUM: pentru `n = 7` și `1125 2 314 101 37 225 12` trebuia `2 37 101`. Afișează `nu exista numere prime`. Pentru **orice** intrare afișează același lucru.

DE CE: bucla citește `x` și îl aruncă — nu-l compară cu nimic, nu-l pune nicăieri. `ct_prime` rămâne `0` de la inițializare, deci `if (ct_prime == 0)` e mereu adevărat. Tot ce ai scris după — sortarea prin selecție și afișarea — e **cod mort**: nu se execută niciodată, indiferent de date. De asta nu-ți dă nicio eroare și pare că „merge”.

Linia care lipsește e legătura dintre `f` și restul. `f(a)` întoarce cel mai mic divizor prim al lui `a`. Un număr e prim exact atunci când cel mai mic divizor prim al lui e **el însuși** — asta e „apelul util” pe care-l cere enunțul.

Al doilea lucru: mesajul. Enunțul cere **NU EXISTA**, tu scrii **nu exista numere prime**. La bac evaluarea e automată și compară textul caracter cu caracter — un mesaj reformulat e 0 puncte pe tot punctul b), chiar cu algoritmul corect.

```
// ACUM
for (int i = 0; i < n; i++) {
    cin >> x;
}
...
cout << "nu exista numere prime";

// CORECT
for (int i = 0; i < n; i++) {
    cin >> x;
    if (f(x) == x) {
        prime[ct_prime] = x;
        ct_prime++;
    }
}
...
cout << "NU EXISTA";
```

B7 — funcție **int** care nu întoarce nimic

varianta39.h:15

CUM E ACUM: warning: „*non-void function does not return a value*”. La rulare pare că merge — **DA / NU** ies corect.

DE CE: antetul **int solutie3()** e o promisiune făcută compilatorului: „la ieșire las un **int** în locul convenit”. Tu nu lași nimic. Standardul numește asta comportament nedefinit — nu „întoarce 0”, ci „orice are voie să se întâmple”. În practică: pe MSVC apelantul citește ce era în registrul de retur, iar compilatoarele care optimizează agresiv au voie să presupună că execuția **nu ajunge** acolo și să taie cod în jur.

Aici nu folosești valoarea de retur, deci merge — dar merge din întâmplare, nu pentru că e corect. Funcția nu întoarce nimic, deci antetul trebuie să spună asta: **void**.

```
// ACUM
int solutie3() {

// CORECT
void solutie3() {
```

B8 — sub e schelet: interclasează și nu afișează nimic

varianta8.h:18–33 (din 4281854 , nu din temă)

CUM E ACUM: `solutie4()` rulează și nu afișează decât o linie goală. `ultima_paritate` e declarat, pus pe `-1` și niciodată citit (warning: „*unused variable*”).

DE CE: ai scris corect **scheletul** interclasării — doi indici, `while (i < n && j < m)`, avansează cel cu valoarea mai mică. Lipsește exact partea care face treaba: în fiecare pas trebuie să **alegi** elementul mai mic, să verifici că paritatea lui diferă de a celui tipărit anterior (asta e rolul lui `ultima_paritate`), și abia apoi să-l tipărești și să actualizezi paritatea. Fără asta, bucla doar consumă cele două tablouri.

E același tipar ca la B6: cadrul e acolo, decizia din mijloc lipsește.

🟡 Importante

M1 — două variante nu pot exista în același proiect

`varianta3.h:40` , `varianta5.h:36` , `varianta8.h:35` , `varianta12.h:48` — toate patru definesc `solutie4()` . Opt fișiere definesc `solutie3()` .

Dovada, pusă la compikator:

```
error: redefinition of 'solutie4'
  varianta5.h:36 | void solutie4() {
note: previous definition is here
  varianta3.h:40 | void solutie4() {
```

Acum merge doar pentru că `varianteBac.cpp` include un singur header. În clipa în care vrei să verifici două variante fără să comentezi linii, proiectul nu se mai construiește. Soluția, în ordinea efortului: prefix în nume (`v3_solutie4` , `v5_solutie4`) sau `namespace v3 { ... }` în fiecare header.

M2 — `sort` definit de două ori și suprapus peste `std::sort`

`varianta1.h:44` și `varianta12.h:28` — aceeași funcție, identică. Pe lângă coliziunea de la M1, numele `sort` intră în concurență cu `std::sort` , adus în scop de `using namespace std;` . Azi câștigă a ta pentru că `<algorithm>` nu e inclus; într-un proiect care îl include, apelul devine ambiguu și eroarea arată complet nelegat de cauză. Numește-o `sortCrescator` .

M3 — prompturile de la tastatură strică evaluarea automată

`varianta1.h:68,71` · `varianta6.h:20` · `varianta11.h:21` · `varianta12.h:54,57`

```
n = introduceți cele 3 numere de câte 5 cifre: 11111 77777
```

La bac, output-ul tău e comparat cu cel așteptat, caracter cu caracter. `n=` în față înseamnă test picat, cu algoritmul perfect. Ține-le cât lucrezi local dacă te ajută — dar fișierul pe care-l consideri „final” nu are voie să le aibă.

M4 — `varianta39` cere citire din fișier

`varianta39.h:21` — enunțul spune explicit „pe prima linie a fișierului `bac.in`”. Tu citești din `cin`. Algoritmul e corect și l-am validat, dar la bac intrarea vine din fișier: `ifstream fin("bac.in");` și apoi `fin >> n`. E singurul lucru care desparte `varianta39` de „complet”.

M5 — subprogramul `nule` afișează, deși n-ar trebui

`varianta20.h:29-31` — enunțul spune că `nule` doar rearanjează și că „tabloul modificat este furnizat tot prin parametrul `a`”. Afișarea e treaba programului principal, nu a subprogramului. Rearanjarea în sine e corectă — verificată prin rulare: `12 -3 -8 0 0 0`. Mută cele trei linii de `cout` în `solutie3`.

Cleanups

- **C1** — `varianta16.h:19` : `a[i] % k` cu `k = 0` e împărțire la zero (comportament nedefinit). Enunțul spune `k` natural, `k ≤ 9` — deci `0` e permis. O gardă `if (k == 0) return 0;` închide subiectul.
 - **C2** — `varianta14.h:22` : `fr[cif]++` fără să verifici că `cif` e între `0` și `9`. O intrare greșită scrie în afara tabloului.
 - **C3** — `varianta5.h:32` : `int f(long a)` primește `long` și întoarce `int`; pe ramura `return a` valoarea se îngustează tăcut. Fă-o `long f(long a)`.
 - **C4** — `#include <cmath>` e în toate cele 14 fișiere și nu e folosit în niciunul. `Header.h` conține doar `#pragma once` — șterge-l.
 - **C5** — marcajele `//??` rămase în `varianta3.h:6`, `varianta5.h:6`, `varianta8.h:6`, `varianta20.h:6`, `varianta24.h:6`. Dacă înseamnă „aici nu sunt sigur”, spune-mi care sunt — le luăm la lecție.
-

✓ Ce e corect — verificat prin rulare

fișier	test	rezultat
varianta1.h (b)	6 / 47 183 69 8 134 -56	DA ✓
varianta1.h (b)	5 / 3 3 1 2 9	NU ✓
varianta7.h	2 / 1 4 5 6 7 8	1 7 5 6 4 8 ✓
varianta12.h	3 / 11111 12345 77777	11111 77777 ✓
varianta16.h	k=3 , {9,273,63,83,93,123}	4 ✓
varianta17.h	{12,27,6,8,9,2}	5 ✓
varianta20.h	{12,0,0,-3,-8,0}	12 -3 -8 0 0 0 ✓
varianta39.h	ambele cazuri	DA / NU ✓

varianta6.h , varianta11.h , varianta14.h — algoritmi corecți, fără obiecții de fond.

Ideea din `varianta1.h` (b) merită spusă: ai rezolvat „sunt toate distincte?” **prin apeluri ale subprogramului** `sub` , așa cum cerea enunțul, nu cu o a doua buclă scrisă pe lângă. Sortezi, apoi verifici că elementul de pe poziția `i` are exact `i` valori strict mai mici înaintea lui. E exact tipul de raționament pentru care se dau punctele la subiectul III.

Q&A — răspunde-mi la astea trei

1. În `varianta24.h`, `x` se transmite ca `int x[]` iar `mini` ca `int mini`. Amândouă sunt „parametri”, dar modificările pe `x` se văd în `solutie2b` și cele pe `mini` nu. **De ce?** Ce primește de fapt funcția în fiecare caz?
2. `varianta3.h` cu `n = 21523531` dă răspunsul corect, iar cu `n = 11009900` dă `1001` în loc de `90011009`. Fără să rulezi nimic: **ce are al doilea număr și primul nu are**, de rupe programul? Și de ce răspunsul poate fi diferit de la o rulare la alta?
3. În `varianta5.h`, sortarea și afișarea sunt scrise corect, dar nu s-au executat niciodată. **Cum ai fi putut afla asta în 30 de secunde**, fără să-mi ceri review? (Indiciu: un singur `cout` pus într-un loc anume.)

Ce ai de făcut, în ordine

1. `varianta24.h` — B1, B2, B3 (sunt în același fișier, se rezolvă împreună).
2. `varianta5.h` — B6: o linie de legătură + mesajul corect.
3. `varianta3.h` — B4, apoi B5.
4. `varianta39.h` — B7 (`int` → `void`) și M4 (`bac.in`).
5. `varianta8.h` — B8, dacă vrei să închizi și restanța din primul push.

M1 (coliziunea de nume) o lăsăm pe runda următoare — nu te blochează acum, dar te va bloca imediat ce vrei să compilezi două variante deodată.

